EVENT-DRIVEN PRECISION

Interrupts, Hardware Timers, and Switch Debouncing on the ESP32

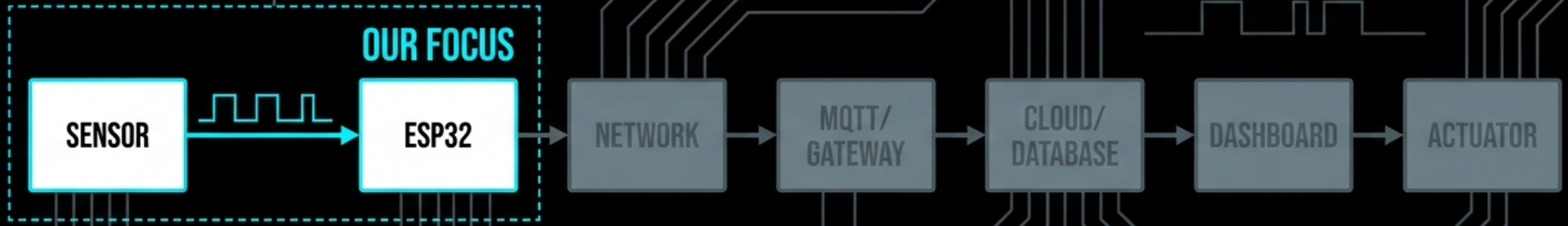
CSE-3079 • EMBEDDED IOT SYSTEMS • WEEK 03

Instructor: Dr. Nasir Mahmood

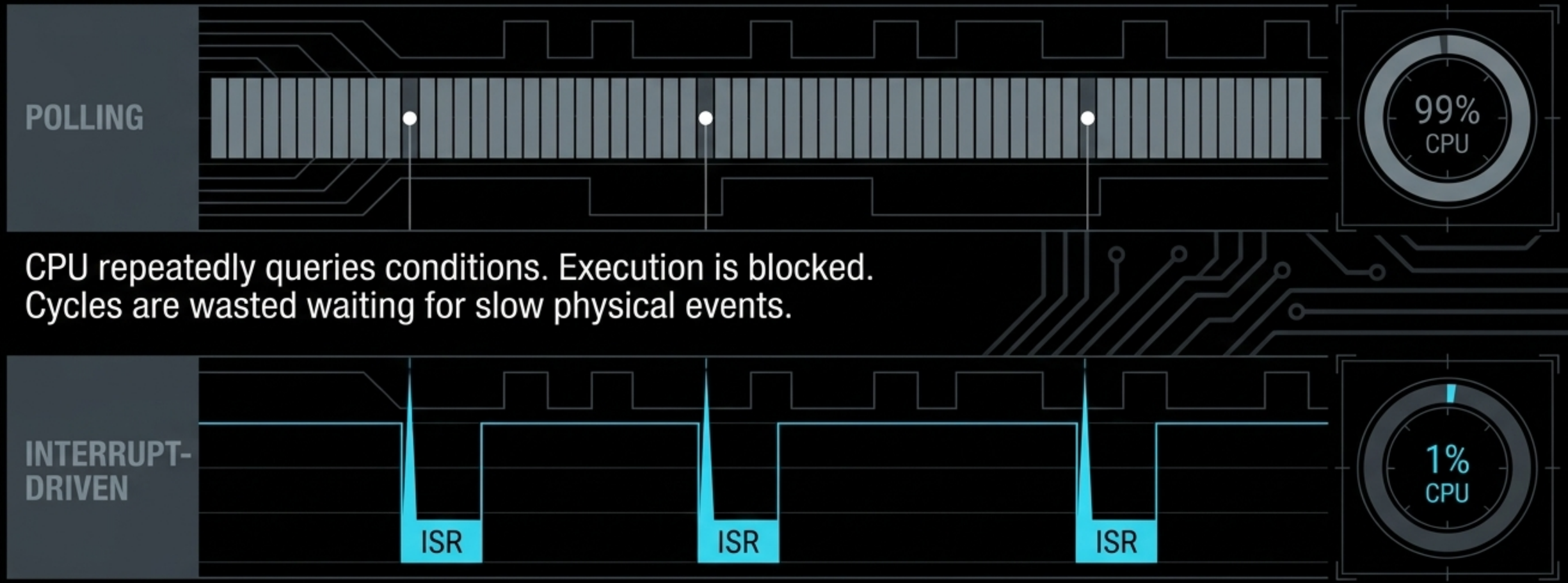
Class : CS 5th A,B

THE END-TO-END IOT ARCHITECTURE

The physical world interacts with the digital world here. Efficiency at the edge dictates system performance.



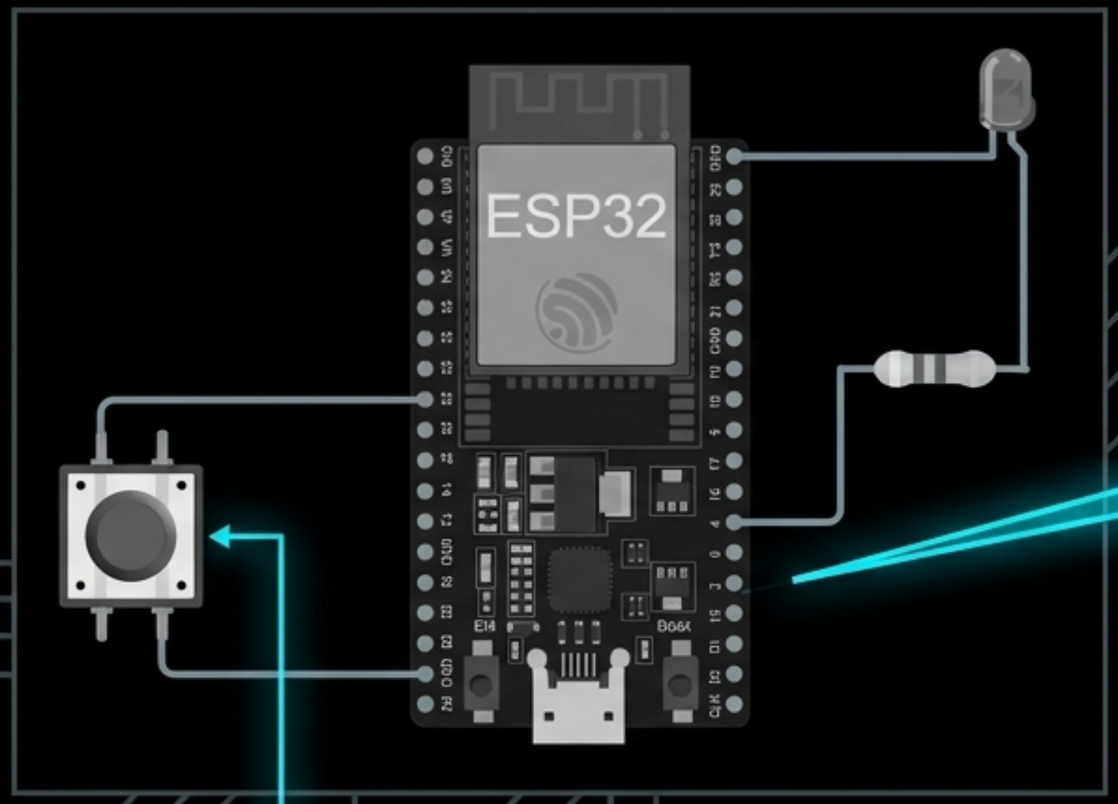
THE CPU BOTTLENECK: POLLING VS. INTERRUPTS

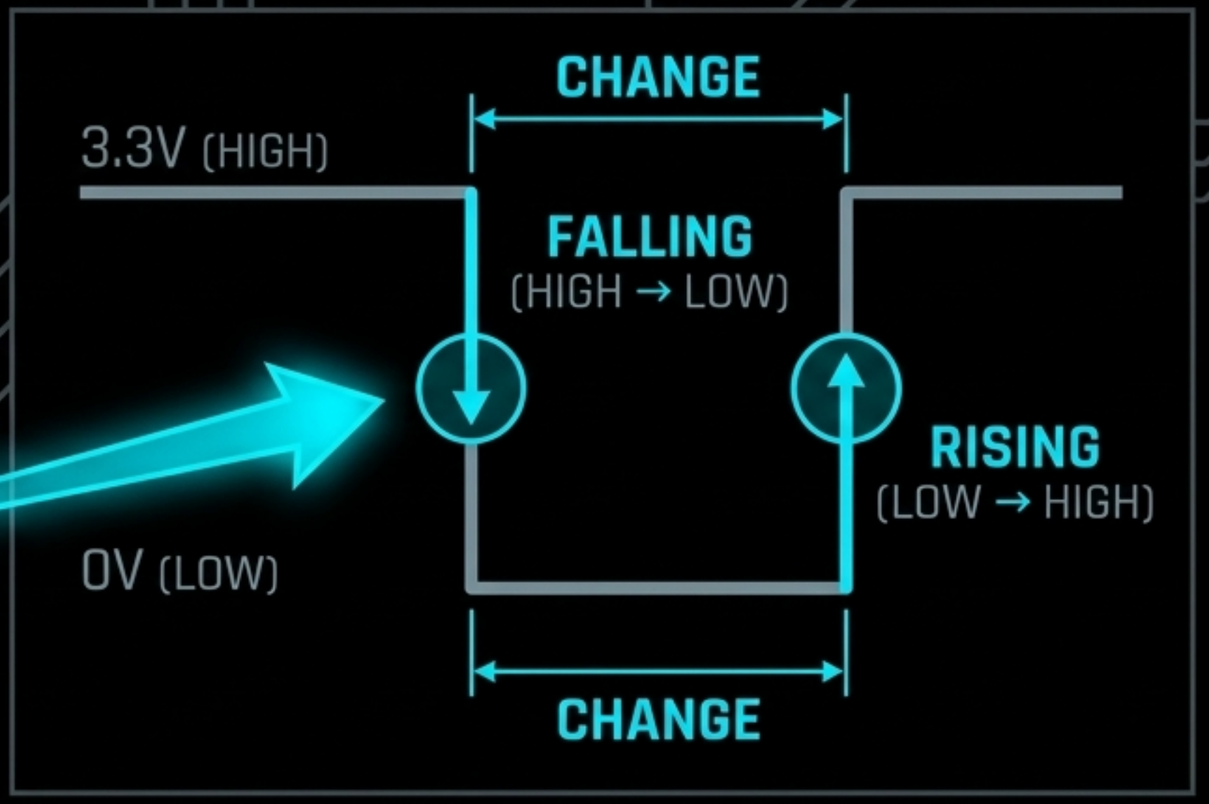


CPU repeatedly queries conditions. Execution is blocked.
Cycles are wasted waiting for slow physical events.

CPU remains asleep or executes primary tasks. Peripherals request service via hardware signals. Zero wasted cycles.

GPIO INTERRUPT MODES & ACTIVE-LOW LOGIC





With **INPUT_PULLUP**, the pin rests **HIGH**. A button press drives it to **GND**. Therefore, a press is a **FALLING** edge.

ATTACHING GPIO INTERRUPTS

```
1  const int buttonPin = 25;
2  volatile bool ledState = LOW;
3
4  // The scream nands ISR into handleButton
5  void IRAM_ATTR handleButton() {
6    ledState = !ledState;
7    digitalWrite(ledPin, ledState);
8  }
9
10 void setup() {
11   pinMode(buttonPin, INPUT_PULLUP);
12   attachInterrupt(digitalPinToInterrupt(buttonPin), handleButton, FALLING);
13 }
```

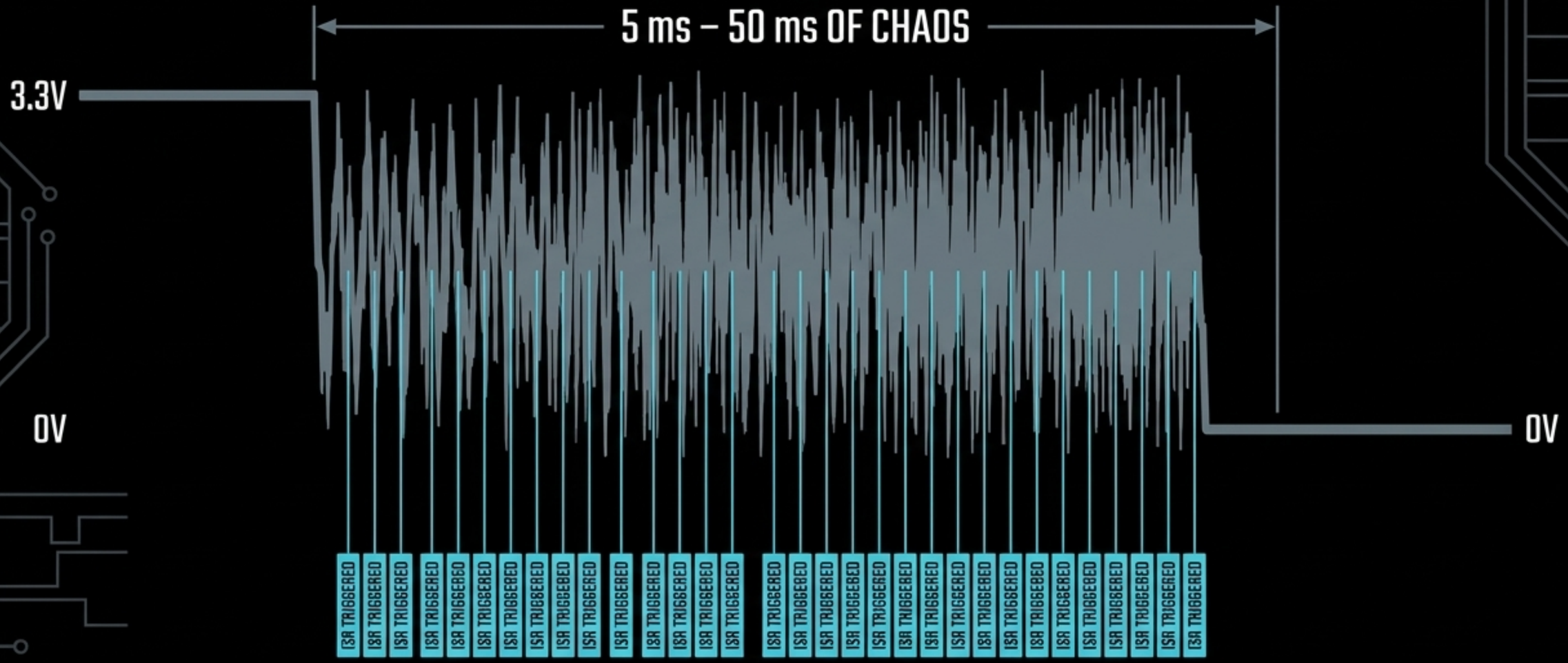
IRAM_ATTR: Forces ISR into Internal RAM for microsecond execution speed.

digitalPinToInterrupt(): Maps Arduino pin to ESP32 silicon routing.

ISR ARCHITECTURE: THE RULES OF EXECUTION

Rule: Execute rapidly and exit.	**Reason:** Minimizes latency. Prevents blocking of concurrent RTOS tasks and other critical hardware interrupts.
Rule: Declare shared variables as volatile.	**Reason:** Prevents compiler optimization. Forces the CPU to fetch the value directly from RAM, as it may mutate asynchronously.
Rule: Never use Serial.print() or delay().	**Reason:** Serial operations and delays are blocking functions. Calling them inside an ISR induces system panics and watch-dog resets.
Rule: Use IRAM_ATTR / ARDUINO_ISR_ATTR.	**Reason:** Pins the instruction block to Internal RAM, preventing cache-miss fatal exceptions when Flash memory is busy.

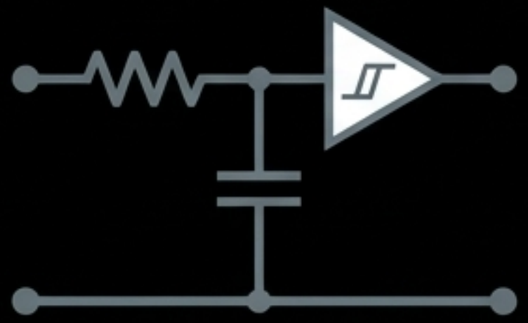
THE PHYSICS COMPLICATION: MECHANICAL BOUNCE



Physical metal contacts act like microscopic springs. One intentional human press generates dozens of rapid electrical transitions.

An interrupt cannot distinguish between human intent and physical physics. It reacts to every edge.

DEBOUNCING ARCHITECTURES



Hardware Filter: Uses an RC circuit and Schmitt trigger to smooth voltage digitally. Highly reliable, but requires extra physical components and footprint.

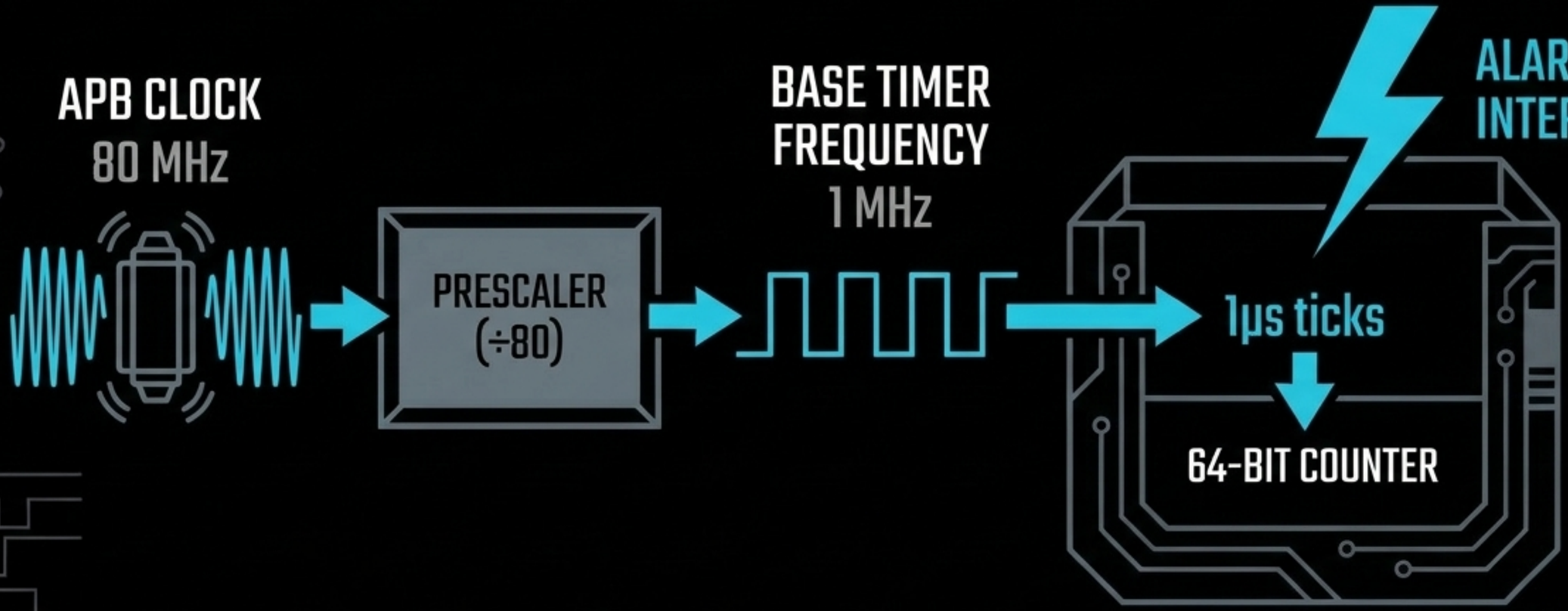


Software Delay: Uses delay(50) in the code to ignore bounces. Simple, but utterly defeats the purpose of non-blocking, event-driven design.



Timer-Based (The Optimal Path): Uses dedicated ESP32 64-bit silicon timers to measure intervals asynchronously. Zero CPU blocking, zero extra hardware.

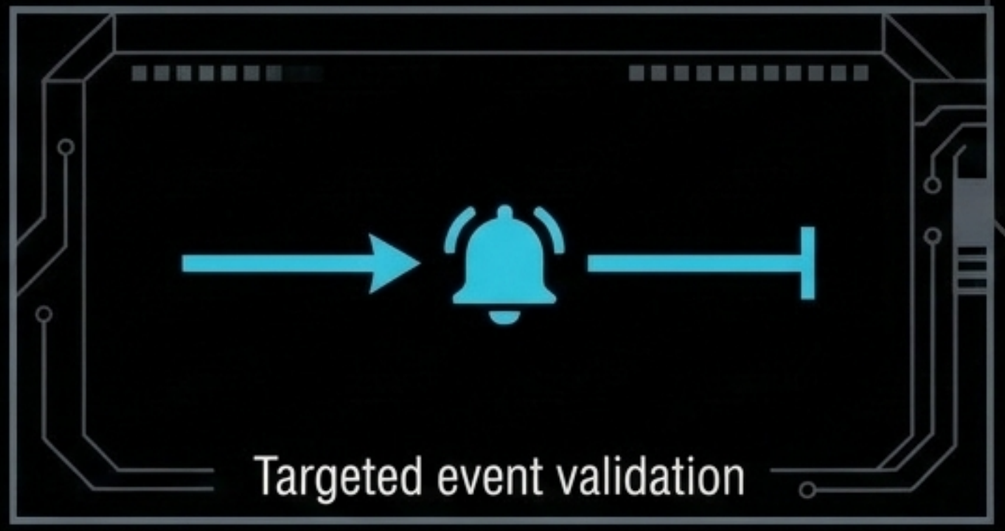
PRECISION CONTROL: 64-BIT HARDWARE TIMERS



ESP32 contains four independent 64-bit hardware timers. They count time in pure silicon, completely decoupled from the main CPU execution loop.

TIMER MODES & TICK MATHEMATICS






```
Period (T) = Alarm Ticks (N) / Frequency (f)

If Frequency = 1,000,000 Hz (1 tick per 1 µs)
And Alarm Ticks = 1,000,000
Then Alarm Triggers Every 1.0 Seconds.
```

Mode Application: We use Auto-Reload for continuous LED blinking, but One-Shot for precise, single-event switch debouncing.

INITIALIZING SILICON TIMERS

```
hw_timer_t *My_timer = NULL;

// 1. Establish Base Frequency (1 MHz = 1µs ticks)
My_timer = timerBegin(1000000);

// 2. Route the Timer to an ISR
timerAttachInterrupt(My_timer, &onTimer);

// 3. Set Alarm Threshold & Mode (1s, Auto-Reload=true)
timerAlarm(My_timer, 1000000, true, 0);
```



Notice there is no edge definition (FALLING/RISING) here. Hardware timers are triggered purely by the accumulation of elapsed time, not physical voltage changes.

THE SYNTHESIS:
NON-BLOCKING TIMER DEBOUNCE



Step 1: GPIO detects the first candidate edge.

Step 2: GPIO ISR arms a 50ms One-Shot Timer and locks out subsequent edges.

Step 3: 50ms elapses. Hardware Timer triggers its own ISR.

Step 4: Timer ISR validates the button is still pressed. State accepted.

THE MASTER ARCHITECTURE: CODE IMPLEMENTATION

```
volatile bool debounceActive = false; // State Guard

void ARDUINO_ISR_ATTR onButtonISR() {
  if (!debounceActive) {
    debounceActive = true;
    // Arm 50ms one-shot alarm
    timerAlarm(debounceTimer, 50000, false, 0);
  }
}

void ARDUINO_ISR_ATTR onDebounceTimer() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    digitalWrite(LED_PIN, !digitalRead(LED_PIN)); // Validated Action
  }
  debounceActive = false; // Unlock for next press
}
```

Takeaway: Zero delay(). Zero wasted CPU cycles. Total resistance to physical noise.

KNOWLEDGE CHECK: TECHNICAL ANSWERS

1. **Primary Advantage:** Interrupts are non-blocking, saving CPU cycles and allowing the processor to perform other tasks while waiting for input.
2. **ISR Execution:** ISRs must be brief to minimize the time the CPU is unavailable for other tasks and to prevent missing other interrupts.
3. `debounceActive` **Check:** This guard prevents multiple triggerings of the timer during the bounce period, ensuring only one debounce sequence runs at a time.
4. **Offloading Delay:** The hardware timer runs independently, allowing the CPU to continue executing the main program without pausing or blocking.
5. `volatile` Keyword: This informs the compiler that the variable's value can be changed unexpectedly by an ISR, preventing compiler optimizations that might use a cached value.